

Multi-Tag Agent System

API Blueprint v1.0

REST API Reference for the Multi-Tag Agent System

Version	1.0
Date	June 17, 2026
Base URL	<code>https://api.multitag-agent.io/v1</code>
Format	REST / JSON
Authentication	Bearer Token (JWT)
Author	Copilot, generated for Emanuel / Auron
Classification	Internal - Developer Reference
Status	Production-Ready

Table of Contents

1. API Overview
2. Authentication & Authorization
3. Error Codes & Error Schema
4. Resource Group: Sessions
5. Resource Group: Tags
6. Resource Group: Messages (Core Routing)
7. Resource Group: IAL – Identity Alias Layer
8. Resource Group: Amplifiers
9. Resource Group: Snapshots & Persistence
10. Resource Group: Health & Diagnostics
11. Data Models & Schemas
12. Rate Limits & Quotas
13. Changelog

1. API Overview

1.1 Introduction

The Multi-Tag Agent System REST API exposes all core capabilities of the agent architecture as stateless HTTP endpoints. It enables external clients — web frontends, CLI tools, CI/CD pipelines, and third-party integrations — to interact with the Tag-based agent system programmatically.

The API is designed around five primary interaction patterns:

1. **Session lifecycle** – create, retrieve, and close agent sessions
2. **Tag management** – register, configure, activate, and deactivate Tags
3. **Message routing** – submit natural-language input and receive routed agent responses
4. **IAL management** – manage user aliases, profiles, and global preferences
5. **Amplifier control** – register custom amplifiers and inspect overlay state

1.2 Base URL

```
https://api.multitag-agent.io/v1
```

All endpoints are relative to this base URL. TLS is mandatory — HTTP requests are rejected with `301 Redirect` to HTTPS.

1.3 Versioning

The API is versioned via URL path (`/v1`). Breaking changes result in a new major version (`/v2`). Minor additions are backward-compatible and documented in the changelog. Deprecated endpoints receive a `Deprecation` response header with the sunset date.

1.4 Content Type

All requests and responses use:

```
Content-Type: application/json Accept: application/json
```

Multipart requests are only used for file uploads (Tag prompt import).

1.5 Idempotency

All mutating requests (`POST`, `PUT`, `PATCH`) accept an optional `Idempotency-Key` header (UUID v4). Requests with the same key within 24 hours return the cached response without re-execution.

2. Authentication & Authorization

2.1 Authentication Method

The API uses JWT Bearer Tokens for authentication.

```
Authorization: Bearer <jwt_token>
```

Tokens are issued via the `/auth/token` endpoint (out-of-scope for this blueprint; handled by the identity provider). Token lifetime: **3600 seconds (1 hour)**. Refresh tokens: **30 days**.

2.2 API Key Authentication (Service-to-Service)

For server-to-server integrations, API Keys are supported as an alternative:

```
X-API-Key: <your_api_key>
```

2.3 Scopes & Permissions

Scope	Permission	Applicable Resources	Notes
agent:read	Read-only access	Sessions, Tags, Snapshots	Default for viewer roles
agent:write	Full CRUD access	Sessions, Tags, Messages	Required for routing
ial:read	Read IAL profile	IAL	Read alias and preferences
ial:write	Mutate IAL profile	IAL	Restricted to admin roles
amplifier:use	Trigger amplifiers	Amplifiers, Messages	Overlay activation
admin:all	Full system access	All resources	Internal use only

2.4 Security Headers (Required on All Requests)

Header	Required	Description
Authorization	Yes (or X-API-Key)	Bearer JWT token
X-API-Key	Yes (or Authorization)	Service-to-service API key
X-Request-ID	Recommended	UUID v4 for request tracing
Idempotency-Key	Optional	UUID v4 for idempotent mutations

3. Error Codes & Error Schema

3.1 Standard Error Response Schema

All errors follow a unified schema:

```
{  "error": {    "code": "TAG_NOT_FOUND",    "message": "The requested Tag with id 'deep_dive' does not exist.",    "details": {      "tag_id": "deep_dive",      "available_tags": ["auron", "dickerchen"]    },    "request_id": "req_01HXYZ1234567890",    "timestamp": "2026-06-17T05:08:00+02:00"  } }
```

3.2 HTTP Status Codes

HTTP Status	Name	Usage
200	OK	Successful read or update
201	Created	Successful resource creation
202	Accepted	Async task accepted (streaming)
204	No Content	Successful delete, no body
400	Bad Request	Malformed request, missing fields
401	Unauthorized	Missing or invalid token

HTTP Status	Name	Usage
403	Forbidden	Valid token, insufficient scope
404	Not Found	Resource does not exist
409	Conflict	Duplicate resource, state conflict
422	Unprocessable Entity	Validation error on request body
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unhandled server error
503	Service Unavailable	LLM backend unavailable

3.3 Application Error Codes

Error Code	HTTP Status	Description
SESSION_NOT_FOUND	404	Session ID does not exist
SESSION_EXPIRED	409	Session TTL exceeded
TAG_NOT_FOUND	404	Tag ID does not exist
TAG_CONFLICT	409	Tag trigger conflicts with existing Tag
TAG_ISOLATION_BREACH	403	Cross-Tag access attempt detected
AMPLIFIER_NOT_FOUND	404	Amplifier keyword not registered
AMPLIFIER_STACK_OVERFLOW	422	Recursion depth exceeded (max: 1)
IAL_WRITE_FORBIDDEN	403	Caller lacks ial:write scope
PROMPT_INJECTION_DETECTED	400	Injection pattern in user_input
MESSAGE_ROUTING_FAILED	500	Context Router could not determine active Tag
SNAPSHOT_NOT_FOUND	404	No snapshot exists for this session/tag
LLM_BACKEND_TIMEOUT	503	LLM API did not respond in time
RATE_LIMIT_EXCEEDED	429	Request quota exhausted

Error Code	HTTP Status	Description
INVALID_TOKEN	401	JWT malformed or expired

4. Resource Group: Sessions

Sessions represent the top-level context container for a user's interaction with the Multi-Tag Agent System. Each session holds the active Tag state, conversation history, IAL reference, and open context snapshots.

4.1 Create Session

POST

/sessions

Creates a new agent session. Returns a session ID and initial state. The Default Tag (Auron Workflow) is activated automatically unless overridden.

Request Headers

Authorization: Bearer <token> Content-Type: application/json Idempotency-Key: <uuid-v4> (optional)

Request Body

```
{  "alias": "Auron",  "default_tag": "auron",  "ial_profile_id": "profile_01HXYZ",  "ttl_seconds": 86400,  "metadata": {    "client": "cli-v1",    "environment": "production"  } }
```

Request Body Fields

Field	Type	Required	Description
alias	string	Yes	User alias (from IAL). Example: "Auron"
default_tag	string	No	Tag ID to activate on

Field	Type	Required	Description
			session start. Default: "auron"
ial_profile_id	string	No	Reference to an existing IAL profile
ttl_seconds	integer	No	Session lifetime in seconds. Default: 86400 (24h)
metadata	object	No	Free-form key-value metadata for client use

Response 201 Created

```
{  "session_id": "sess_01HXYZ1234567890",  "alias": "Auron",  "active_tag": "auron",  "ial_profile_id": "profile_01HXYZ",  "created_at": "2026-06-17T05:08:00+02:00",  "expires_at": "2026-06-18T05:08:00+02:00",  "status": "active",  "metadata": {    "client": "cli-v1",    "environment": "production"  } }
```

4.2 Retrieve Session

GET

/sessions/{session_id}

Returns the current state of an existing session including active Tag, open tasks, and amplifier state.

Path Parameters

Parameter	Type	Required	Description
session_id	string	Yes	Session ID returned by POST /sessions

Response 200 OK

```
{  "session_id": "sess_01HXYZ1234567890",  "alias": "Auron",  "active_tag": "auron",  "amplifier_active": false,  "open_tasks": [],
```



```
"last_activity": "2026-06-17T05:08:00+02:00",    "expires_at": "2026-06-18T05:08:00+02:00",    "status": "active",    "message_count": 12 }
```

4.3 List Sessions

GET

`/sessions`

Returns all active sessions for the authenticated user.

Query Parameters

Parameter	Type	Required	Description
status	string	No	Filter: "active", "expired", "closed". Default: "active"
limit	integer	No	Max results. Default: 20, Max: 100
offset	integer	No	Pagination offset. Default: 0

Response 200 OK

```
{  "sessions": [    {      "session_id": "sess_01HXYZ1234567890",      "alias": "Auron",      "active_tag": "auron",      "status": "active",      "created_at": "2026-06-17T05:08:00+02:00",      "expires_at": "2026-06-18T05:08:00+02:00"    }  ],  "total": 1,  "limit": 20,  "offset": 0 }
```

4.4 Close Session

DELETE

`/sessions/{session_id}`

Closes a session and writes a final context snapshot to the Persistence Layer.
Session data is retained for 30 days before permanent deletion (GDPR-compliant).

NOTE

Response:

204 No Content

— no response body is returned.

4.5 Extend Session TTL

PATCH

`/sessions/{session_id}`

Extends the TTL of an active session.

Request Body

```
{  "extend_by_seconds": 3600 }
```

Response 200 OK

```
{  "session_id": "sess_01HXYZ1234567890",  "new_expires_at": "2026-06-18T06:08:00+02:00" }
```

5. Resource Group: Tags

Tags are the core prompt containers of the system. Each Tag encapsulates a system prompt, behavioral rules, trigger keywords, and isolation settings. Tags can be listed, created (custom), activated, deactivated, and deleted.

5.1 List Available Tags

GET

/tags

Returns all registered Tags for the authenticated user, including system defaults and custom Tags.

Query Parameters

Parameter	Type	Required	Description
type	string	No	Filter: "system", "custom", "all". Default: "all"
active	boolean	No	Filter to only currently active Tags

Response 200 OK

```
{  "tags": [    {      "tag_id": "auron",      "name": "Auron Workflow",      "type": "system",      "is_default": true,      "isolation_level": "high",      "amplifier_compatible": true,      "trigger_keywords": ["plan", "task", "schedule", "workflow"],      "created_at": "2026-06-17T00:00:00Z"    },    {      "tag_id": "deep_dive",      "name": "Deep-Dive Technical",      "type": "system",      "is_default": false,      "isolation_level": "full",      "amplifier_compatible": true,      "trigger_keywords": ["code", "architecture", "debug", "deep-dive"],      "created_at": "2026-06-17T00:00:00Z"    }  ],  "total": 2 }
```

5.2 Get Tag Details

GET

/tags/{tag_id}

Returns full details of a single Tag including the system prompt (redacted for non-admin callers).

Response 200 OK

```
{  "tag_id": "auron",  "name": "Auron Workflow",  "type": "system",  "is_default": true,  "isolation_level": "high",  "amplifier_compatible": true,  "trigger_keywords": ["plan", "task", "schedule", "workflow"],  "verbosity": "medium",  "output_format": "structured",  "task_tracking": true,  "system_prompt": "[REDACTED - admin:all scope required]",  "version": "1.0",  "created_at": "2026-06-17T00:00:00Z",  "updated_at": "2026-06-17T00:00:00Z" }
```

5.3 Create Custom Tag

POST

/tags

Registers a new custom Tag with a user-defined system prompt, trigger keywords, and isolation level.

Request Body

```
{  "tag_id": "creative",  "name": "Creative Mode",  "system_prompt": "You are a creative ideation assistant for Auron. Focus on brainstorming, lateral thinking, and unconventional solutions...",  "trigger_keywords": ["brainstorm", "ideas", "creative", "imagine"],  "isolation_level": "high",  "amplifier_compatible": true,  "is_default": false,  "output_format": "freeform",  "verbosity": "high",  "metadata": {    "author": "Auron",    "project": "IdeaEngine v2"  } }
```

Request Body Fields

Field	Type	Required	Description
tag_id	string	Yes	Unique identifier. Lowercase, underscores only. Max 32 chars

Field	Type	Required	Description
name	string	Yes	Human-readable Tag name
system_prompt	string	Yes	Full system prompt text. Max 8,192 characters
trigger_keywords	array[string]	Yes	List of natural-language trigger words. Min 1, Max 20
isolation_level	string	No	"high" or "full". Default: "high"
amplifier_compatible	boolean	No	Whether Dickerchen overlay applies. Default: true
is_default	boolean	No	Set as fallback Tag. Only one default allowed. Default: false
output_format	string	No	"structured", "freeform", "technical". Default: "structured"
verbosity	string	No	"low", "medium", "high". Default: "medium"
metadata	object	No	Free-form key-value pairs

Response 201 Created

```
{  "tag_id": "creative",  "name": "Creative Mode",  "type": "custom",  "is_default": false,  "created_at": "2026-06-17T05:08:00+02:00",  "trigger_conflict_check": {    "status": "passed",    "conflicts": []  } }
```

5.4 Update Tag

PUT

/tags/{tag_id}

Replaces a custom Tag's configuration entirely. System Tags (auron, deep_dive) are immutable.

Request Body: Same schema as POST /tags (all fields required). **Response 200**

OK: Updated Tag object.

5.5 Patch Tag

PATCH

/tags/{tag_id}

Partially updates a custom Tag's configuration. Only provided fields are updated.

Request Body Example (partial)

```
{  "trigger_keywords": ["brainstorm", "ideas", "creative", "imagine",  
"invent"],  "verbosity": "medium" }
```

Response 200 OK: Updated Tag object.

5.6 Delete Custom Tag

DELETE

/tags/{tag_id}

Deletes a custom Tag. System Tags cannot be deleted. If the Tag is currently active in a session, the session falls back to the Default Tag.

NOTE

Response:

204 No Content

— no response body is returned.

5.7 Activate Tag in Session

POST

`/sessions/{session_id}/tags/{tag_id}/activate`

Explicitly switches the active Tag in a given session. Triggers a context snapshot of the previously active Tag. Equivalent to an explicit natural-language trigger but executed programmatically.

Response 200 OK

```
{  "session_id": "sess_01HXYZ1234567890",  "previous_tag": "auron",  "active_tag": "deep_dive",  "snapshot_id": "snap_01HXYZ9876543210",  "switched_at": "2026-06-17T05:08:00+02:00" }
```

5.8 Get Active Tag in Session

GET

`/sessions/{session_id}/tags/active`

Returns the currently active Tag for a given session, including amplifier state.

Response 200 OK

```
{  "session_id": "sess_01HXYZ1234567890",  "active_tag": {    "tag_id": "auron",    "name": "Auron Workflow",    "amplifier_active": false,    "amplifier_id": null  } }
```

6. Resource Group: Messages (Core Routing)

The Messages resource is the heart of the Multi-Tag Agent API. It accepts natural-language input, routes it through the Context Router, applies IAL enrichment and optional amplifier overlays, calls the LLM backend, and returns a structured response. This is the primary integration point for all client applications.

6.1 Send Message (Core Routing Endpoint)

POST

/sessions/{session_id}/messages

Sends a natural-language message to the system. The Context Router automatically detects Tag triggers and amplifier keywords in the input. The active Tag processes the message and returns a structured response.

Request Body

```
{  "user_input": "Deep-Dive: Analyze the trade-offs between monolithic and microservice architectures for a real-time data pipeline.",  "override_tag": null,  "amplifier_override": null,  "stream": false,  "metadata": {    "source": "cli",    "client_timestamp": "2026-06-17T05:08:00+02:00"  } }
```

Request Body Fields

Field	Type	Required	Description
user_input	string	Yes	Natural-language input. Max 32,768 characters

Field	Type	Required	Description
override_tag	string	No	Force a specific Tag ID, bypassing router detection
amplifier_override	string	No	Force-apply an amplifier keyword
stream	boolean	No	Enable SSE streaming response. Default: false
metadata	object	No	Client-side metadata for logging and tracing

Response 200 OK (Non-Streaming)

```
{  "message_id": "msg_01HXYZ1122334455",  "session_id":
"sess_01HXYZ1234567890",  "routed_to_tag": "deep_dive",  "tag_switched":
true,  "previous_tag": "auron",  "amplifier_active": false,
"snapshot_created": "snap_01HXYZ9876543211",  "response": {    "content":
"// Deep-Dive active\n\nMONOLITH vs. MICROSERVICES - Real-Time Data Pipeline\n
n-----\nMONOLITH:\n+ Simpler
deployment; single process\n+ Low inter-service latency\n- Scaling bottleneck
at high throughput\n- Single point of failure\n\nMICROSERVICES:\n+
Independent scaling per component\n+ Fault isolation; resilience\n- Network
overhead; eventual consistency\n- Operational complexity (service mesh,
observability)\n\nRECOMMENDATION [context: real-time pipeline]:\n
nMicroservices preferred for > 10k events/sec; monolith viable below that
threshold.",    "output_format": "technical",    "task_status_snapshot":
null,    "amplifier_addition": null  },    "tokens_used": {      "prompt":
312,      "completion": 198,      "total": 510    },    "latency_ms": 1240,
"created_at": "2026-06-17T05:08:01+02:00" }
```

6.2 Send Message - Streaming (SSE)

POST

/sessions/{session_id}/messages

with "stream": true

When streaming is enabled, the response is delivered as Server-Sent Events (SSE). The connection stays open until the full response has been streamed.

Response Content-Type

text/event-stream

SSE Event Format

```
event: message_start data: {"message_id": "msg_01HXYZ1122334455",
"routed_to_tag": "deep_dive"} event: content_delta data: {"delta": "// Deep-
Dive active\n\n"} event: content_delta data: {"delta": "MONOLITH vs.
MICROSERVICES"} event: message_end data: {"tokens_used": {"prompt": 312,
"completion": 198, "total": 510}, "latency_ms": 1240}
```

SSE Event Types

Event Type	Description
message_start	Fired once; contains routing metadata
content_delta	Incremental content chunk
tag_switch	Fired if a Tag switch occurs mid-stream
amplifier_start	Fired when amplifier overlay begins
amplifier_end	Fired when amplifier addition is complete
message_end	Fired once; contains token usage and final metadata
error	Error event; connection closed after this event

6.3 Get Message History

GET

/sessions/{session_id}/messages

Returns the full conversation history for a session, ordered chronologically.

Query Parameters

Parameter	Type	Required	Description
limit	integer	No	Max messages to return. Default: 50,

Parameter	Type	Required	Description
			Max: 200
offset	integer	No	Pagination offset. Default: 0
tag_filter	string	No	Filter messages by Tag ID
since	string	No	ISO 8601 timestamp; return messages after this time

Response 200 OK

```
{  "session_id": "sess_01HXYZ1234567890",  "messages":
[    {      "message_id": "msg_01HXYZ1122334455",      "role": "user",
"content": "Deep-Dive: Analyze trade-offs...",      "tag": "deep_dive",
"amplifier_active": false,      "created_at": "2026-06-17T05:08:00+02:00"
},    {      "message_id": "msg_01HXYZ5566778899",      "role":
"assistant",      "content": "// Deep-Dive active\n\nMONOLITH vs...",
"tag": "deep_dive",      "amplifier_active": false,      "tokens_used":
510,      "created_at": "2026-06-17T05:08:01+02:00"    }  ],  "total": 2,
"limit": 50,  "offset": 0 }
```

6.4 Get Single Message

GET

`/sessions/{session_id}/messages/{message_id}`

Returns the full details of a single message including routing decisions and amplifier additions.

Response 200 OK: Full message object as shown in POST /messages response above.

7. Resource Group: IAL – Identity Alias Layer

The Identity Alias Layer (IAL) stores the persistent user identity that is injected into every Tag context. It contains the alias, communication preferences, and global behavioral settings that travel across all Tag switches.

7.1 Get IAL Profile

GET

`/ial/profiles/{profile_id}`

Returns the full IAL profile for a user.

Response 200 OK

```
{  "profile_id": "profile_01HXYZ",  "alias": "Auron",  "communication_style": "direct",  "preferred_output_format": "structured",  "preferred_verbosity": "medium",  "active_projects": ["Project Alpha", "Release v2.1"],  "global_context": "User prefers numbered steps and time estimates in all workflow outputs.",  "language": "en",  "timezone": "Europe/Berlin",  "created_at": "2026-06-17T00:00:00Z",  "updated_at": "2026-06-17T05:08:00+02:00" }
```

7.2 Create IAL Profile

POST

`/ial/profiles`

Creates a new IAL profile. Required scope: ial:write.

Request Body

```
{  "alias": "Auron",  "communication_style": "direct",  "preferred_output_format": "structured",  "preferred_verbosity": "medium",  "active_projects": ["Project Alpha"],  "global_context": "User prefers numbered steps and time estimates.",  "language": "en",  "timezone": "Europe/Berlin" }
```

IAL Profile Fields

Field	Type	Required	Description
alias	string	Yes	Persistent user alias. Immutable after creation. Max 64 chars
communication_style	string	No	"direct", "formal", "casual". Default: "direct"
preferred_output_format	string	No	"structured", "freeform", "technical". Default: "structured"
preferred_verbosity	string	No	"low", "medium", "high". Default: "medium"
active_projects	array[string]	No	List of current project names injected into Tag context
global_context	string	No	Free-text context injected into every Tag. Max 1,024 chars
language	string	No	ISO 639-1 language code. Default: "en"
timezone	string	No	IANA timezone name. Default: "UTC"

Response 201 Created: Full IAL profile object.

7.3 Update IAL Profile

PATCH

/ial/profiles/{profile_id}

Partially updates an IAL profile. The `alias` field is immutable and cannot be patched.
Required scope: `ial:write`.

Response 200 OK: Updated profile object.

7.4 Delete IAL Profile

DELETE

`/ial/profiles/{profile_id}`

WARNING

Permanently deletes an IAL profile and all associated sessions and snapshots (GDPR right to erasure). This action is irreversible.

NOTE

Response:

204 No Content

— no response body is returned.

8. Resource Group: Amplifiers

Amplifiers are overlay modifiers that can be applied to any active Tag. The built-in amplifier is **Dickerchen** (keyword: `dickerchen`). Custom amplifiers can be registered. All amplifiers operate in single-shot mode by default.

8.1 List Amplifiers

GET

`/amplifiers`

Returns all registered amplifiers.

Response 200 OK

```
{  "amplifiers": [    {      "amplifier_id": "dickerchen",      "keyword": "dickerchen",      "type": "system",      "depth_multiplier": 2.0,      "add_alternatives": true,      "add_risks": true,      "persistence": "single-shot",      "created_at": "2026-06-17T00:00:00Z"    }  ],  "total": 1 }
```

8.2 Create Custom Amplifier

POST

`/amplifiers`

Registers a custom amplifier with user-defined overlay behavior.

Request Body

```
{  "amplifier_id": "turbo",  "keyword": "turbo",  "overlay_prompt": "TURBO MODE: Respond with maximum speed and brevity. Bullet points only. No elaboration.",  "depth_multiplier": 0.5,  "add_alternatives": false,  "add_risks": false,  "persistence": "single-shot" }
```

Request Body Fields

Field	Type	Required	Description
amplifier_id	string	Yes	Unique identifier. Lowercase, underscores. Max 32 chars
keyword	string	Yes	Natural-language trigger keyword. Case-insensitive
overlay_prompt	string	Yes	Overlay instruction added to active Tag prompt. Max 2,048 chars
depth_multiplier	float	No	Output depth scaling factor. Default: 2.0
add_alternatives	boolean	No	Instruct model to include alternatives. Default: true
add_risks	boolean	No	Instruct model to include risks/edge cases. Default: true
persistence	string	No	"single-shot" or "session". Default: "single-shot"

Response 201 Created: Full amplifier object.

8.3 Get Amplifier State in Session

GET

/sessions/{session_id}/amplifiers/state

Returns the current amplifier state for a session.

Response 200 OK


```
{  "session_id": "sess_01HXYZ1234567890",  "amplifier_active": false,
  "last_amplifier_used": "dickerchen",  "last_activated_at": "2026-06-17T04:55:00+02:00",
  "activation_count": 3 }
```

8.4 Manually Trigger Amplifier

POST

`/sessions/{session_id}/amplifiers/{amplifier_id}/trigger`

Programmatically activates an amplifier for the next message, without requiring the keyword in the `user_input`.

Response 200 OK

```
{  "session_id": "sess_01HXYZ1234567890",  "amplifier_id": "dickerchen",
  "status": "queued",  "applies_to_next_message": true }
```

9. Resource Group: Snapshots & Persistence

Snapshots are JSON-serialized captures of a Tag's state at a specific point in time. They are created automatically on every Tag switch and can be created manually. Snapshots enable lossless context restoration across Tag switches and sessions.

9.1 List Snapshots for Session

GET

`/sessions/{session_id}/snapshots`

Returns all snapshots created in a session.

Query Parameters

Parameter	Type	Required	Description
tag_filter	string	No	Filter snapshots by Tag ID
limit	integer	No	Default: 20, Max: 100
offset	integer	No	Default: 0

Response 200 OK

```
{  "snapshots": [    {      "snapshot_id": "snap_01HXYZ9876543210",      "session_id": "sess_01HXYZ1234567890",      "tag_id": "auron",      "trigger": "tag_switch",      "open_tasks": ["Release plan v2.1 - step 3 pending"],      "last_output_summary": "Project week plan created",      "created_at": "2026-06-17T05:08:00+02:00"    }  ],  "total": 1 }
```

9.2 Get Snapshot Detail

GET

`/sessions/{session_id}/snapshots/{snapshot_id}`

Returns the full snapshot including serialized context data.

Response 200 OK

```
{  "snapshot_id": "snap_01HXYZ9876543210",  "session_id": "sess_01HXYZ1234567890",  "tag_id": "auron",  "alias": "Auron",  "trigger": "tag_switch",  "open_tasks": ["Release plan v2.1 - step 3 pending"],  "last_output_summary": "Project week plan created",  "message_count_at_snapshot": 8,  "conversation_excerpt": "[last 3 messages serialized]",  "created_at": "2026-06-17T05:08:00+02:00" }
```

9.3 Create Manual Snapshot

POST

`/sessions/{session_id}/snapshots`

Creates a manual snapshot of the current Tag state.

Request Body

```
{  "tag_id": "auron",  "label": "Pre-release checkpoint" }
```

Response 201 Created: Snapshot object.

9.4 Restore Snapshot

POST

`/sessions/{session_id}/snapshots/{snapshot_id}/restore`

Restores a session to the state captured in the snapshot. The active Tag is switched to the snapshot's Tag and context is loaded. A backup snapshot of the current state is created automatically before restoration.

Response 200 OK

```
{  "session_id": "sess_01HXYZ1234567890",  "restored_snapshot_id": "snap_01HXYZ9876543210",  "backup_snapshot_id": "snap_01HXYZBACKUP0001",  "active_tag": "auron",  "restored_at": "2026-06-17T05:10:00+02:00" }
```

9.5 Delete Snapshot

DELETE

`/sessions/{session_id}/snapshots/{snapshot_id}`

Permanently deletes a snapshot. Auto-generated tag-switch snapshots cannot be deleted (system-managed). Only manually created snapshots can be deleted.

NOTE

Response:

204 No Content

— no response body is returned.

10. Resource Group: Health & Diagnostics

10.1 Health Check

GET

/health

Returns the operational status of all system components. No authentication required for basic liveness check.

Response 200 OK

```
{  "status": "healthy",  "timestamp": "2026-06-17T05:08:00+02:00",
"components": {    "api": "healthy",    "context_router": "healthy",
"llm_backend": "healthy",    "redis": "healthy",    "database": "healthy"
},  "version": "1.0.0",  "uptime_seconds": 86400 }
```

Response 503 Service Unavailable

```
{  "status": "degraded",  "components": {    "api": "healthy",
"context_router": "healthy",    "llm_backend": "unavailable",    "redis":
"healthy",    "database": "healthy"  } }
```

10.2 Router Diagnostics - Dry Run

POST

/diagnostics/route

Simulates the Context Router's Tag detection logic for a given input without creating a session or calling the LLM backend. Useful for testing trigger keyword coverage and amplifier detection.

Request Body

```
{  "user_input": "Dickerchen, Deep-Dive: analyze my schema.",  "current_tag": "auron" }
```

Response 200 OK

```
{  "input": "Dickerchen, Deep-Dive: analyze my schema.",  "detected_tag": "deep_dive",  "tag_switch_required": true,  "previous_tag": "auron",  "amplifier_detected": true,  "amplifier_id": "dickerchen",  "trigger_type": "explicit",  "matched_keywords": ["deep-dive"],  "matched_amplifier_keywords": ["dickerchen"],  "conflict": false,  "routing_confidence": 0.98 }
```

Response Fields

Field	Type	Description
detected_tag	string	Tag ID the router would activate
tag_switch_required	boolean	Whether a Tag switch would occur
amplifier_detected	boolean	Whether an amplifier keyword was detected
trigger_type	string	"explicit", "implicit", or "default"
routing_confidence	float	Router confidence score (0.0–1.0)
conflict	boolean	Whether a keyword conflict was detected

10.3 Audit Log

GET

`/diagnostics/audit`

Returns the audit log for Tag switches, amplifier activations, and security events.

Requires `admin:all` scope.

Query Parameters

Parameter	Type	Required	Description
<code>session_id</code>	string	No	Filter by session
<code>event_type</code>	string	No	"tag_switch", "amplifier_activation", "security_event"
<code>since</code>	string	No	ISO 8601 timestamp
<code>limit</code>	integer	No	Default: 100, Max: 1,000

Response 200 OK

```
{  "events": [    {      "event_id": "evt_01HXYZ1234567890",      "session_id": "sess_01HXYZ1234567890",      "event_type": "tag_switch",      "from_tag": "auron",      "to_tag": "deep_dive",      "trigger_type": "explicit",      "amplifier_active": true,      "timestamp": "2026-06-17T05:08:00+02:00"    }  ],  "total": 1 }
```

11. Data Models & Schemas

11.1 Complete Object Reference

Object	Description	Key Fields
Session	Top-level context container	session_id, alias, active_tag, status, expires_at
Tag	Prompt container and behavioral unit	tag_id, name, system_prompt, trigger_keywords, isolation_level
Message	Single user/assistant turn	message_id, role, content, routed_to_tag, amplifier_active
IAL Profile	Persistent user identity	profile_id, alias, global_context, preferences
Amplifier	Overlay modifier	amplifier_id, keyword, depth_multiplier, persistence
Snapshot	Serialized Tag state	snapshot_id, tag_id, open_tasks, last_output_summary
AuditEvent	Log entry for system events	event_id, event_type, from_tag, to_tag, timestamp
Error	Standard error response	code, message, details, request_id, timestamp

11.2 Enumerations

isolation_level

"high" | "full"

- high: No cross-Tag prompt access; IAL data shared (read-only)
- full: Complete isolation; no shared data except session_id

output_format

"structured" | "freeform" | "technical"

verbosity

"low" | "medium" | "high"

communication_style

"direct" | "formal" | "casual"

trigger_type

"explicit" | "implicit" | "default" | "override"

amplifier_persistence

"single-shot" | "session"

session_status

"active" | "expired" | "closed"

12. Rate Limits & Quotas

12.1 Rate Limit Tiers

Tier	Requests / Minute	Tokens / Day	Sessions (concurrent)	Notes
Free	20	100,000	1	For development/testing
Standard	100	1,000,000	10	Default for registered users
Pro	500	10,000,000	50	For power users
Enterprise	Unlimited	Unlimited	Unlimited	SLA-backed; custom contract

12.2 Rate Limit Headers

Returned on every response:

Header	Description
X-RateLimit-Limit	Maximum requests allowed per minute
X-RateLimit-Remaining	Remaining requests in current window
X-RateLimit-Reset	Unix timestamp when the window resets
Retry-After	Seconds to wait after a 429 response

12.3 Payload Size Limits

Field	Limit
user_input	32,768 characters
system_prompt (Tag)	8,192 characters
overlay_prompt (Amplifier)	2,048 characters
global_context (IAL)	1,024 characters
Conversation history (per session)	200 messages

13. Changelog

Version	Date	Changes
1.0	2026-06-17	Initial release. All core resource groups: Sessions, Tags, Messages, IAL, Amplifiers, Snapshots, Health. Full CRUD for all custom resources. Streaming (SSE) support. Router Diagnostics dry-run endpoint.
1.1 (planned)	TBD	Semantic embedding-based router; webhook support for async message delivery; multi-user IAL namespacing.
2.0 (planned)	TBD	Multimodal input (image/audio); stacked amplifiers (stable); GraphQL alternative interface.

Multi-Tag Agent System - API Blueprint v1.0

Created: June 17, 2026 | For: Emanuel (Auron) | Status: **Production-Ready** |
Classification: Internal – Developer Reference

Generated by Copilot. All endpoint specifications, payload schemas, and enumeration values are defined within this document. This blueprint supersedes all prior informal specifications.